

몽글몽글

Monggle Monggle

지나간 기록을 꺼내 오늘의 글로 잇는 시스템

통합 설계 문서



과거에 했던 일이 미래의 내가 기억을 되살려볼 때

몽글몽글 생각 나도록

Index

01 Overview	1장 프로젝트 개요 2장 문제 정의 3장 사용자 시나리오
02 Architecture	4장 전체 아키텍처 5장 AI 허브 분리 6장 외부 통합 및 업·다운로드
03 Data & Permission	7장 데이터 모델 8장 권한 모델 9장 미디어 저장
04 Resource Design	10장 이벤트 소싱 + 시점 스냅샷 11장 버퍼 / 큐 / 메모리 관리 12장 트래픽 처리 + 캐시 다층 + 팬아웃
05 Operation	13장 API 명세 14장 AWS 인프라 운영 계획 15장 부하 테스트 계획

1장. 프로젝트 개요

1.1 한 줄 정의

"과거에 했던 일이 미래의 내가 기억을 되살려볼 때 몽글몽글 생각 나도록."

사용자가 직접 기록한 일지·사진·영상·코멘트를 시점 기반으로 저장하고, 나중에 시점을 임의로 복원·비교·재활용할 수 있는 시스템이다. 라인/카카오톡과 유사한 사용자 흐름 위에, 이벤트 소싱 기반 시점 복원과 다중 사용자 트래픽 처리를 핵심 어필 축으로 둔다.

1.2 시작 질문

"내가 지난달에 무엇을 했는지, 왜 이 결정을 했는지 기억나지 않을 때, 시스템은 그 시점을 어떻게 다시 떠올리게 해줄 수 있는가?"

개발일지를 일일이 쓰는 것이 귀찮았다. 이미 했던 일은 글·사진·코드로 흩어져 있는데, 한 달 뒤·세 달 뒤의 내가 그 시점을 다시 보고 싶어도 검색이 잘 되지 않았다. 단순한 메모장은 "오늘 무엇을 썼는가"는 알려주지만, "어떤 시점에 내가 어떤 상태였는가"는 알려주지 못한다. 몽글몽글은 그 간극을 다루는 시스템이다.

1.3 핵심 어필 축 3가지

- 시점 복원 사고 — 이벤트 소싱 + 주기적 스냅샷으로 임의 시점의 사용자 상태를 복원한다.
- 자원 설계 사고 — 버퍼·큐·메모리 워터마크·백프레셔를 명시적으로 설계한다.

- 트래픽 처리 사고 — 다중 사용자 환경의 팬아웃·캐시 다층·미디어 다운로드 부하를 다룬다.

2장. 문제 정의

2.1 풀고자 하는 진짜 문제

6개월 전의 나와 지금의 나는 생각하는 방식도, 코드를 보는 시야도 다르다. 기록하지 않으면 과거의 내가 어떤 의도로 결정을 내렸는지 지금의 나는 알 수 없다.

개발을 시작할 때 기획서나 개발일지에 의도를 적는 일은, 단순히 "써야 한다"는 환경적 요구 때문만은 아니다. 시간이 흐른 뒤 그 기록을 다시 보면, 그때의 결정이 어떤 맥락에서 최선이었는지 보이고, 지금의 내가 그 위에 한층 더 발전시킬 수 있는 비교군이 된다. 과거의 기록은 현재의 성장을 가능하는 기준이 된다.

문제는 두 가지다. 첫째, 개발 중 떠오른 의도와 맥락을 글로 정리하는 일 자체가 부담스럽다. 결국 다른 도구나 AI를 참고하게 되지만, 매번 처음부터 초안을 써내는 일은 여전히 막막하다. 둘째, 이전에 작성한 기록과 작업물을 다시 꺼내 보는 일이 어렵다. 한 달이 지나면 키워드 검색조차 잘 되지 않고, 그때의 사진·코드·메모가 흩어져 있어 시점을 통째로 다시 떠올릴 방법이 없다.

몽글몽글은 이 두 가지를 같이 다룬다. 사용자가 남긴 기록을 시점 기반으로 저장하여 언제든지 다시 보고 다운받을 수 있게 하고, 동시에 새 일지를 작성할 때는 그 시점의 활동을 바탕으로 초안을 제공한다.

2.2 기존 도구의 한계

도구 유형	잘 다루는 것	다루지 못하는 것
메모장 / Notion	텍스트 작성·검색	시점 단위 복원, 의미 기반 검색, 미디어와 텍스트의 연결
블로그 / 티스토리 / 카페	글 작성·공개·검색	수정 이력 보존, 시점 단위 상태 복원, 본인 자료 통합 다운로드, AI 기반 초안 보조

도구 유형	잘 다루는 것	다루지 못하는 것
사진 앨범	사진 시점 정렬	사진과 함께 있던 컨텍스트(코드·메모)
Git 로그	코드 변경 이력	변경의 의도, 같은 시점의 비코드 활동
SNS (인스타·트위터)	공개 기록, 친구 공유	본인 자료 다운로드/재활용, 시점 비교

특히 블로그·카페는 "기록과 공유"는 잘 풀지만 "과거 시점의 그때 상태를 정확히 복원"하는 기능이 없다. 글을 수정하면 이전 버전은 사라지고, 본인이 올린 자료를 한꺼번에 다운로드해 다시 활용하는 흐름도 지원하지 않는다. 작성 시 AI가 그 시점의 활동을 바탕으로 초안을 제공하는 기능도 없다.

2.3 시스템이 풀어야 할 것

- 사용자가 직접 작성한 글·사진·영상·코멘트·코드 스니펫을 시점과 함께 저장한다.
- 임의 시점(예: "2026년 3월 15일")으로 사용자의 상태를 복원해 그 시점 전후 콘텐츠를 보여준다.
- 키워드 정확도가 낮은 검색(예: "Redis 처음 만진 날")을 의미 기반 임베딩 검색으로 처리한다.
- 두 시점을 비교한다(예: "3개월 전 vs 지금 자주 다룬 주제 변화").
- 본인 자료는 무제한 다운로드/재활용 가능하게 한다.
- 친구(팔로우 관계)와 선택적으로 공유한다 (전체/친구만/개인만).

3장. 사용자 시나리오

3.1 페르소나

주 사용자는 "매일 무언가를 만드는 사람"이다. 개발자, 디자이너, 학습자가 1차 타겟이며, 본 문서에서는 개발자 사용자를 기준으로 시나리오를 기술한다.

3.2 시나리오 A — 일과 기록

사용자가 작업 중 의미 있는 순간을 기록하는 흐름이다.

- HTTP 클라이언트(웹)에서 "새 일지" 버튼을 누른다.
- 텍스트로 "오늘 Redis Pub/Sub로 알림 시스템 다시 짤. SUBSCRIBE 패턴이 PSUBSCRIBE보다 명시적이라 디버깅이 쉬웠음." 작성한다.
- 디버깅 화면 스크린샷 2장과 짧은 화면 녹화 영상 1개를 첨부한다.
- 코드 스니펫 블록을 추가한다.
- 공개 범위를 "친구만"으로 설정하고 발행한다.

이때 백엔드는 글 본문은 즉시 저장하고, 미디어는 청크 업로드 큐에 적재해 백그라운드로 처리하며, 임베딩 생성은 AI 허브 큐에 별도로 적재한다. 사용자는 발행 응답을 즉시 받고, 임베딩·썸네일은 비동기로 채워진다.

3.3 시나리오 B — 시점 복원

사용자가 한 달 전 시점을 다시 보고 싶을 때의 흐름이다.

- "한 달 전" 또는 임의 날짜를 선택한다.
- 시스템이 그 시점에 가장 가까운 스냅샷을 로드한다.
- 스냅샷 이후 발생한 이벤트를 재생해 정확한 시점 상태를 만든다.
- 그 시점 전후 ± 3 일의 글·미디어를 시간 순으로 보여준다.

3.4 시나리오 C — 의미 기반 검색

키워드가 부정확한 검색이다.

- 사용자가 "Redis 처음 만진 날"을 검색한다.
- 백엔드는 검색어를 임베딩 벡터로 변환한다 (AI 허브 호출, 결과 캐시).
- 사용자 본인 콘텐츠 임베딩 인덱스에서 코사인 유사도 상위 N개를 가져온다.
- 그 중 가장 "이른" 시점의 글을 후보로 제시한다.

3.5 시나리오 D — 시점 비교

- "3개월 전 vs 지금" 비교를 요청한다.
- 두 시점의 사용자 활동 임베딩을 클러스터링한다.
- 주제 분포 차이를 시각화한다 ("Spring Boot 비중 ↓, Redis 비중 ↑").
- LLM이 차이를 자연어로 설명한다 (외부 API 호출, 결과 캐시).

3.6 시나리오 E — 본인 자료 재활용

사용자가 과거 자료를 다시 다운로드해 활용하는 흐름이다. 본 시스템의 핵심 가치 중 하나로, 본인 자료는 항상 원본 다운로드 가능하다.

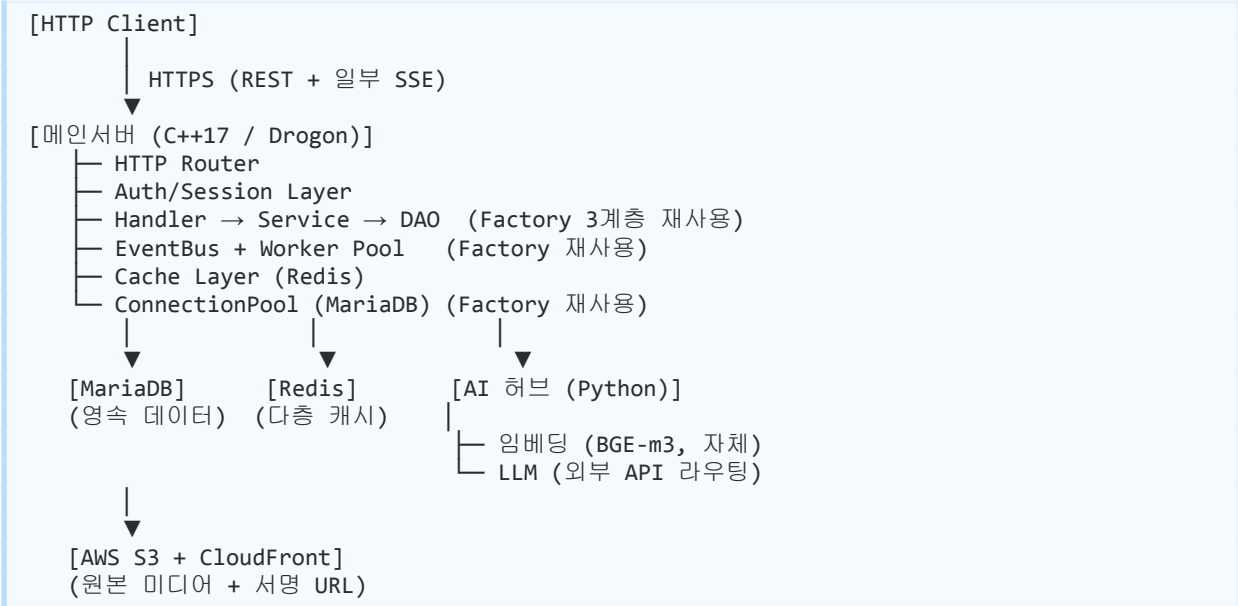
- "내 자료 일괄 다운로드 (월별)" 요청을 한다.
- 백엔드가 다운로드 작업을 큐에 적재한다 (Factory의 작업 큐 패턴).
- 워커가 해당 월의 글·미디어를 zip으로 묶는다.
- 완료되면 알림과 함께 서명 URL(만료 24시간)을 발급한다.

3.7 시나리오 F — 친구 자료 보기

- 팔로우한 친구의 "전체 공개" 또는 "친구만" 글 피드를 본다.
- 미디어는 시청 가능, 다운로드는 작성자 정책에 따라 제한된다.
- 이 흐름이 다중 사용자 트래픽의 핵심 발생 지점이다 (13장 참고).

4장. 전체 아키텍처

4.1 전체 그림



4.2 컴포넌트 책임

컴포넌트	책임	Factory 대응
HTTP Router	REST 엔드포인트 라우팅	(신규) GUI Listener와 유사 위치
Handler 계층	요청 파싱·검증·변환	Factory Handler와 동일 패턴
Service 계층	비즈니스 로직·트랜잭션	Factory Service와 동일
DAO 계층	DB 접근 (Prepared Statement)	Factory DAO와 동일
EventBus	비동기 이벤트 발행/구독	Factory EventBus 그대로
Worker Pool	이벤트 처리 워커 (4~8 스레드)	Factory Worker Pool 그대로
AI 허브	임베딩·LLM 요청 큐 처리	Factory AI Server 분리 패턴
Cache Layer	다층 캐시 (피드/세션/임베딩)	(신규)

4.3 Hub-and-Spoke 원칙

Factory와 마찬가지로, 모든 데이터 흐름은 메인서버를 통과한다. 클라이언트는 AI 허브와 직접 통신하지 않으며, S3에 대한 직접 업로드도 메인서버가 발급한 서명 URL을 통해서만 이루어진다. 이 원칙의 이유는 Factory와 동일하다 — 장애 발생 시 추적 가능성, 데이터 일관성, 보안 정책의 단일 적용 지점.

4.4 통신 프로토콜

구간	프로토콜	비고
Client ↔ 메인서버	HTTPS (REST + SSE)	Factory의 TCP+JSON 커스텀 프로토콜과 다른 결. 일반 웹 표준.
메인서버 ↔ AI 허브	내부 HTTP/gRPC (선택)	내부망 통신, 인증 토큰 기반
메인서버 ↔ MariaDB	MariaDB Connector/C++	Factory와 동일
메인서버 ↔ Redis	redis-cpp 또는 hiredis	(신규)
메인서버 ↔ S3	aws-sdk-cpp	(신규)

5장. AI 허브 분리

5.1 분리 근거

Factory에서 AI 서버를 메인서버에서 분리한 핵심 이유는 세 가지였다 — (a) AI 모델 로딩이 무거워 메인서버 재시작에 영향을 주면 안 됨, (b) AI 추론이 메인서버 스레드를 잡으면 일반 요청이 느려짐, (c) 모델 교체를 메인서버 재배포 없이 하려면 분리가 필수. 이 세 이유는 몽글몽글에도 그대로 적용된다.

5.2 AI 허브 책임

기능	모델 / 방식	호스팅
텍스트 임베딩	BGE-m3 (다국어, 1024차원)	자체 호스팅 (CPU 가능)
LLM 생성·요약	Claude Haiku 또는 GPT-4o-mini	외부 API 라우팅
임베딩 캐시 관리	Redis 또는 자체 인메모리	AI 허브 내부
요청 큐	asyncio.Queue (Factory와 동일)	AI 허브 내부

5.3 메인서버 → AI 허브 호출 흐름

```
[메인서버]
  EventBus: POST_PERSISTED 발행
    ↓
  EmbeddingRequester (구독자)
    ↓ HTTP POST (idempotency_key 포함)
[AI 허브]
  asyncio.Queue 적재
    ↓
  Worker가 BGE-m3 호출 (CPU 또는 GPU)
    ↓ HTTP POST (callback URL)
[메인서버]
  EmbeddingIndexer가 수신 → DB 인덱스 갱신
```

5.4 외부 API 라우팅 — LLM의 자원 관점 처리

LLM은 외부 API라 비용·지연·실패 가능성이 모두 자원 문제다. AI 허브 내부에서 다음 정책을 적용한다:

- 호출 큐 + 배치 — 사용자가 시점 비교를 요청해도 즉시 LLM 호출하지 않고, 짧은 시간(예: 200ms) 내 들어온 요청을 묶어 한 번에 처리
- 결과 캐시 — 같은 입력의 LLM 호출은 N분 동안 캐시
- 회로 차단 (Circuit Breaker) — 외부 API가 N회 연속 실패 시 일정 시간 호출 차단, fallback 메시지 반환
- 타임아웃 + 재시도 — 외부 API는 타임아웃 30초, 지수 백오프 재시도 (Factory의 1→5→30→120→300초 패턴 적용)

5.5 모델 버전 관리

Factory의 모델 바이너리 TCP 중계와는 다른 결로, 임베딩 모델의 버전이 바뀌면 기존 임베딩 인덱스가 무효화된다. 다음 정책을 둔다:

- 임베딩 레코드는 `model_version` 필드를 함께 저장
- 모델 업그레이드 시 신/구 버전을 일정 기간 병행
- 백그라운드 워커가 구 버전 임베딩을 점진적으로 재계산

6장. 외부 통합 및 업·다운로드

6.1 통합 대상

대상	용도	신뢰성 정책
AWS S3	원본 미디어 저장	서명 URL 만료, 먹등 업로드 키
AWS CloudFront	미디어 다운로드 가속	서명 쿠키 또는 서명 URL
외부 LLM API	시점 비교 자연어 설명, 자동 태깅	회로 차단, 캐시, 배치
Git API (선택)	사용자가 명시적으로 연결한 저장소의 최근 커밋 리스트	rate limit 준수, 명시적 동의 필요

6.2 미디어 업로드 흐름

클라이언트가 메인서버를 거쳐 S3에 직접 업로드하는 두 단계 패턴을 사용한다. 메인서버 자체가 미디어 바이트를 받지 않는 이유는 자원 효율 — 큰 파일이 메인서버 메모리/네트워크를 점유하지 않게 한다.

1. 클라이언트: POST /uploads (메타데이터: 파일명, 크기, MIME)
2. 메인서버: 검증 → S3 멀티파트 서명 URL 발급 → DB에 upload_pending 레코드
3. 클라이언트: S3로 직접 청크 업로드 (재개 가능)
4. 클라이언트: POST /uploads/{id}/complete
5. 메인서버: S3 메타 검증 → upload_completed → MEDIA_UPLOADED 이벤트 발행
6. ThumbnailGenerator(워커): 사진은 썸네일, 영상은 첫 프레임 추출

6.3 청크 업로드 + 재개 가능

큰 영상 파일(예: 200MB)을 모바일에서 업로드하다 네트워크가 끊기면 처음부터 다시 올리는 것은 사용자 경험상 치명적이다. S3 멀티파트 업로드 자체가 청크/재개를 지원하므로 이를 활용한다.

- 청크 크기: 5MB (S3 멀티파트 최소 권장)
- upload_id를 DB에 저장해 재개 가능
- 미완료 업로드는 24시간 후 S3 LifeCycle 규칙으로 자동 정리

6.4 다운로드 흐름

다운로드는 권한 모델(9장)을 거친 후 서명 URL을 발급한다.

1. 클라이언트: GET /media/{id}/download
2. 메인서버: 권한 체크 (본인/친구/공개) → 작성자 다운로드 정책 체크
3. 통과 시: CloudFront 서명 URL 발급 (만료 1시간)
4. 클라이언트: 서명 URL로 다운로드

6.5 본인 자료 일괄 다운로드

"내 자료 월별 일괄 다운로드"는 Factory의 큐 패턴을 가장 자연스럽게 활용하는 기능이다.

1. 클라이언트: POST /me/archives { period: "2026-03" }
2. 메인서버: idempotency_key 기반 중복 방지 → 외부 큐에 적재 → 202 Accepted
3. ArchiveBuilder 워커:
 - 해당 월 글/미디어 메타데이터 수집
 - 메모리 워터마크 체크 (12장)
 - S3에서 스트리밍으로 zip 생성 (전체 메모리 적재 X)
 - 완성된 zip을 별도 S3 경로에 저장 (lifecycle 7일)
4. ARCHIVE_READY 이벤트 → Notifier가 사용자에게 알림 + 서명 URL

7장. 데이터 모델

7.1 핵심 원칙 — Raw vs 발행 분리

이 시스템 데이터 모델의 핵심 원칙은 사용자가 직접 작성한 발행 콘텐츠와 시스템이 자동 수집/생성한 **raw** 데이터를 분리하는 것이다. 이유는 두 가지다 — **(a)** 자동 수집 데이터는 절대 외부 공개되지 않아야 하고, **(b)** 권한 모델이 다르다(**raw**는 무조건 본인만, 발행은 사용자 선택).

7.2 주요 엔티티

엔티티	설명	공개 가능 여부
users	사용자 계정 (인증·프로필)	프로필 일부 공개 가능
follows	팔로우 관계 (단방향)	팔로워 목록은 본인만 전체 조회
posts	사용자 발행 글	공개 범위(전체/친구/개인) 사용자 선택
media_assets	미디어 메타데이터 (S3 키 포함)	posts에 종속, 동일 정책
post_events	글 관련 이벤트 (작성·수정·삭제)	본인만 (시점 복원용 raw)
activity_events	외부 자동 수집 활동 (Git 등)	절대 비공개
snapshots	주기적 사용자 상태 스냅샷	본인만
embeddings	텍스트 임베딩 인덱스	본인 검색용, 비공개
downloads	다운로드 권한 정책	본인 설정
archives	일괄 다운로드 작업 상태	본인만

7.3 posts 테이블 — 발행 콘텐츠

```
posts (
  id          BIGINT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  body        TEXT NOT NULL,
  visibility   ENUM('public','friends','private') NOT NULL,
  download_policy ENUM('owner_only','followers','public_allowed') NOT NULL,
  created_at   DATETIME(3) NOT NULL,
  updated_at   DATETIME(3) NOT NULL,
  embedding_id BIGINT NULL,
  INDEX idx_user_created (user_id, created_at),
  INDEX idx_visibility_created (visibility, created_at),
  FOREIGN KEY (user_id) REFERENCES users(id)
)
```

7.4 post_events 테이블 — 시점 복원용 이벤트 소스

이 테이블이 시점 복원의 근거다. `posts`의 현재 상태가 아니라, 변경 이력 자체를 저장한다.

11장에서 자세히 다룬다.

```
post_events (
  id          BIGINT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  post_id     BIGINT NOT NULL,
  event_type   ENUM('created','edited','deleted','visibility_changed','media_added') NOT NULL,
  payload_json JSON NOT NULL,
  occurred_at  DATETIME(3) NOT NULL,
  INDEX idx_user_time (user_id, occurred_at),
  INDEX idx_post_time (post_id, occurred_at)
)
```

7.5 snapshots 테이블 — 주기적 스냅샷

```
snapshots (
  id          BIGINT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  taken_at    DATETIME(3) NOT NULL,
  state_json   JSON NOT NULL,          -- 그 시점 사용자 상태 요약
  event_cursor BIGINT NOT NULL,        -- 마지막 반영 post_events.id
  INDEX idx_user_time (user_id, taken_at)
)
```

7.6 follows 테이블 — 단방향 팔로우

```
follows (
  follower_id BIGINT NOT NULL,
  followee_id  BIGINT NOT NULL,
  created_at   DATETIME(3) NOT NULL,
  PRIMARY KEY (follower_id, followee_id),
  INDEX idx_followee (followee_id)
)
```

7.7 인덱싱 전략

- posts: (user_id, created_at) — 본인 타임라인 조회
- posts: (visibility, created_at) — 전체 공개 피드
- post_events: (user_id, occurred_at) — 시점 복원 시 핵심 인덱스
- follows: (followee_id) — 팬아웃 시 팔로워 조회
- embeddings: pgvector 사용 시 ivfflat 인덱스, 그렇지 않으면 별도 벡터 인덱스 서비스

8장. 권한 모델

8.1 두 축으로 구성된 권한

권한은 두 축의 조합으로 결정된다

— (a) 콘텐츠의 공개 범위 (전체/친구/개인), (b) 보는 사람과 작성자의 관계 (본인/팔로워/타인).

8.2 조회 권한 매트릭스

공개 범위 \ 보는 사람	본인	팔로워	타인
전체 공개	조회 가능	조회 가능	조회 가능
친구만	조회 가능	조회 가능	차단
개인만	조회 가능	차단	차단

8.3 다운로드 권한 매트릭스

작성자가 다운로드 정책을 별도로 설정한다. 본인은 항상 다운로드 가능하다.

다운로드 정책 \ 보는 사람	본인	팔로워	타인 (전체 공개 시)
owner_only (기본)	가능	보기만	보기만
followers	가능	가능	보기만
public_allowed	가능	가능	가능

8.4 권한 체크 위치

권한 체크는 두 곳에서 모두 일어난다 — **Service** 계층(메타데이터 조회 시)과 미디어 게이트웨이(서명 URL 발급 시). 두 곳에서 모두 막아야 "메타는 차단되지만 직접 미디어 URL로 접근하면 보임" 같은 우회를 막는다.

8.5 서명 URL 정책

- 조회용 서명 URL: 만료 1시간
- 다운로드용 서명 URL: 만료 24시간 + Content-Disposition: attachment 강제
- 아카이브 다운로드: 만료 24시간, lifecycle 7일 후 zip 자체 삭제
- 타인 자료의 직접 URL을 다른 사람에게 공유해도, 만료된 후엔 작동하지 않음

8.6 raw 데이터 격리

activity_events·post_events·snapshots·embeddings는 어떤 공개 범위에서도 외부에 노출되지 않는다. 이 테이블에 접근하는 모든 API는 필수적으로 본인 user_id 검증을 거친다. 데이터 모델 단계에서 "공개 가능 여부" 필드 자체가 없다.

9장. 미디어 저장

9.1 저장 정책

미디어 종류	저장 방식	비고
사진 (jpg/png/webp)	원본 + 썸네일 2종 (200px, 800px)	썸네일은 백그라운드 생성
영상 (mp4/mov/webm)	원본만 저장	인코딩 없음. 첫 프레임만 추출하여 썸네일로 활용
코드 스니펫	텍스트로 DB 저장	미디어 아님, <code>posts.body</code> 의 일부
외부 영상 URL	URL과 oEmbed 메타만 저장	YouTube/Vimeo 등 임베드

9.2 영상에 대한 의도적 결정

영상은 자체 인코딩·트랜스코딩·HLS/DASH 스트리밍을 의도적으로 제외한다. 이유:

- 자체 호스팅 트랜스코딩은 별도 시스템 규모 (FFmpeg 워커, GPU, CDN 통합, 세그먼테이션)
- 이 시스템의 어필은 "미디어 처리"가 아니라 "시점 복원과 자원 설계"
- 브라우저가 직접 재생 가능한 포맷(.mp4 H.264)을 사용자에게 권장하면 충분

9.3 S3 키 구조

원본: s3://monggle-media/users/{user_id}/posts/{post_id}/{asset_id}/original
 썸네일: s3://monggle-media/users/{user_id}/posts/{post_id}/{asset_id}/thumb_200
 첫프레임: s3://monggle-media/users/{user_id}/posts/{post_id}/{asset_id}/poster
 아카이브: s3://monggle-archive/users/{user_id}/{period}/archive_{job_id}.zip

9.4 미디어 메타데이터

```
media_assets (
  id          BIGINT PRIMARY KEY,
  post_id     BIGINT NOT NULL,
  user_id     BIGINT NOT NULL,
  kind        ENUM('photo', 'video', 'external_embed') NOT NULL,
  s3_key_original  VARCHAR(512) NULL,
  s3_key_thumb   VARCHAR(512) NULL,
  s3_key_poster  VARCHAR(512) NULL,
```

```
external_url    VARCHAR(1024) NULL,  
mime_type      VARCHAR(100) NULL,  
size_bytes     BIGINT NULL,  
width_px       INT NULL,  
height_px      INT NULL,  
duration_ms    INT NULL,  
status         ENUM('pending','ready','failed') NOT NULL,  
created_at     DATETIME(3) NOT NULL  
)
```

9.5 사진 썸네일 워커

MEDIA_UPLOADED 이벤트를 받아 백그라운드에서 처리한다. 이미지 라이브러리는

OpenCV(Factory에서 사용)나 ImageMagick. CPU 자원 사용을 통제하기 위해 Media Pool(2 스레드)에서만 처리한다.

9.6 영상 첫 프레임 워커

영상 인코딩은 안 하지만, 첫 프레임 추출은 한다 (피드에 정지 이미지로 보여주기 위해).

FFmpeg를 시스템 명령으로 호출하되, 출력은 단일 PNG. 처리 시간이 길어질 수 있으므로 Heavy Pool에 배치.

10장. 이벤트 소싱 + 시점 스냅샷

10.1 왜 이 시스템에 시점 복원이 필요한가

일반 SNS는 "지금 이 사람의 글 목록"만 알려주면 충분하다. 그러나 몽글몽글의 핵심 가치는 "3개월 전 그때의 내가 무엇을 했는지"를 다시 보여주는 것이다. 단순한 `SELECT * WHERE created_at < '...'` 으로는 이 가치를 만들 수 없다.

개발일지는 SNS와 다르다. 사용자는 같은 글을 여러 번 수정한다. "Redis 처음 만진 날" 글을 한 달 뒤 다시 열어 더 자세히 적기도 하고, 공개 범위를 바꾸기도 한다. 그래서 현재 DB의 `posts` 테이블은 "가장 최근 상태"만 알 뿐, 그 사이 어떤 변화가 있었는지 모른다. "3개월 전 그때 그 글"을 보여주려면 변경 이력 자체가 데이터로 남아 있어야 한다.

즉 이 시스템은 단순히 글을 저장하는 것이 아니라, "글의 시간 자체"를 저장한다. 이게 일반 클라우드 메모장과 결정적으로 다른 점이다.

10.2 이벤트 소싱 구조

`posts`의 현재 상태와는 별도로, 모든 변경을 `post_events`에 누적한다. 임의 시점 `t`의 사용자 상태는 "t보다 이른 모든 이벤트를 누적 적용한 결과"다.

```
post_events
- created      글 생성
- edited       본문 수정
- deleted      글 삭제 (soft)
- visibility_changed  공개 범위 변경
- media_added  미디어 첨부
```

→ `posts`는 "현재 상태"만 보여주는 캐시
→ `post_events`는 "변화의 진실" 그 자체

10.3 시점 조회의 비용 문제

이벤트 소싱의 약점은 시점 조회 비용이다. 사용자가 1년치 활동에서 100,000개 이벤트를 누적했다면, "3개월 전 시점"을 보려면 100,000개 이벤트를 처음부터 재생해야 한다. 사용자 한 명의 시점 조회 한 번에 100,000번의 메모리 연산. 비현실적이다.

해결은 주기적 스냅샷이다. 일정 간격마다 "그 시점까지 누적한 상태"를 snapshots에

저장해 두면, 임의 시점 조회 시:

- 그 시점 이전의 가장 가까운 스냅샷을 로드
- 스냅샷 이후 ~ 그 시점까지의 이벤트만 재생

이 방식이 자원 설계의 핵심이다. 스냅샷이 없으면 조회 비용이 $O(\text{전체 이벤트})$, 스냅샷이 있으면 $O(\text{스냅샷 간격})$ 으로 떨어진다. 100,000개 이벤트가 1,000개로 줄어든다.

10.4 스냅샷 정책

정책	값	근거
스냅샷 주기	사용자별 매일 1회 + 1,000 이벤트마다 1회	조회 비용을 $O(1\text{일치 이벤트})$ 로 보장
스냅샷 보관	최근 30일 일별 + 그 이전 월별 1개	오래된 시점은 정밀도 낮춰도 무방
스냅샷 압축	JSON gzip 압축	저장 공간 절약, CPU 트레이드오프 수용
가비지 컬렉션	사용자 삭제 시 cascade	데이터 삭제 권리 보장

10.5 임의 시점 복원 알고리즘

```

fn restore_state(user_id, target_time) -> UserState:
    # 1. target_time 이전의 가장 가까운 스냅샷 로드
    snap = db.query("""
        SELECT * FROM snapshots
        WHERE user_id = ? AND taken_at <= ?
        ORDER BY taken_at DESC LIMIT 1
    """, user_id, target_time)

    if snap is None:
        state = empty_state()
        cursor = 0
    else:
        state = decompress_json(snap.state_json)
        cursor = snap.event_cursor

    # 2. 스냅샷 이후 ~ target_time까지 이벤트만 재생
    events = db.query("""
        SELECT * FROM post_events
        WHERE user_id = ? AND id > ? AND occurred_at <= ?
        ORDER BY occurred_at ASC
    """, user_id, cursor, target_time)
    
```



```
for ev in events:
    state = apply_event(state, ev)

return state
```

10.6 스냅샷 무결성 — Atomic Write

Factory Hub의 Atomic File Write 패턴을 스냅샷 생성에도 적용한다. 스냅샷 생성 중 워커가 죽어도 부분 스냅샷이 저장되지 않게, 트랜잭션 안에서 **INSERT**하고 끝나야만 **commit**한다. 부분 스냅샷이 들어가면 그 이후 모든 시점 복원이 깨지기 때문이다.

10.7 시점 검색의 변형

"3개월 전" 같은 상대 시점은 사용자의 가입일 기준으로 절대 시점으로 변환한다. "내 첫 글 이후 100번째 시점" 같은 이벤트 카운트 기반 시점도 같은 알고리즘으로 풀린다 — `event_cursor`를 시간 대신 `cursor` 값으로 비교.

11장. 버퍼 / 큐 / 메모리 관리

11.1 자원 설계의 위치

이 시스템에서 메모리·버퍼·큐는 다음 4지점에 명시적으로 등장한다.

- HTTP 요청 수신 버퍼 — 큰 multipart 요청의 처리
- EventBus 내부 큐 — 발행 속도 > 처리 속도일 때의 백프레서
- 미디어 업로드 — 메모리에 올리지 않고 S3로 직접 스트리밍
- 아카이브 빌더 — 큰 zip을 메모리에 적재하지 않고 스트리밍 생성

11.2 HTTP 요청 크기 한계

Factory에서 "JSON 64KB, 이미지 50MB, 모델 500MB 상한"을 둔 것과 같은 결의 정책을 둔다.

요청 종류	상한	근거
일반 JSON 요청	256KB	글 본문 + 메타데이터 총분
사진 직접 업로드 (S3 미사용 폴백)	10MB	사용 안 함이 기본, 안전망
S3 멀티파트 청크	16MB	5MB 권장 청크 + 마진
메모리에 올리지 않는 스트리밍	제한 없음	S3 직접 업로드는 메모리 비점유

11.3 EventBus 큐 백프레서

EventBus의 인프로세스 큐가 무한히 커지면 메모리 폭발이 온다. 큐에 워터마크를 둔다.

워터마크	값	동작
High Water (HW)	10,000건	신규 발행 거부 (혹은 외부 큐로 강제 이동)
Low Water (LW)	5,000건	HW 진입 후 이 값으로 떨어지면 발행 재개

Critical	20,000건	HEALTH 알림 발행, 운영자 개입 필요
----------	---------	-------------------------

HW 도달 시의 정책은 이벤트 종류에 따라 다르다. 사용자 가시

이벤트(`POST_VALIDATED`)는 거부 시 사용자 응답 실패로 이어지므로, 외부 큐(`Redis`

`Stream`)로 우회. 내부 후처리 이벤트(`MEDIA_UPLOADED`)는 발행 자체를 짧게 지연(`100ms` 백오프).

11.4 미디어 업로드 메모리 비점유

핵심 원칙은 "메인서버 프로세스 안에 미디어 바이트를 절대 올리지 않는다"이다. 7장에서 설명한 `S3` 직접 업로드 패턴이 이 원칙의 실현이다. 메인서버는 메타데이터(파일명, 크기, `MIME`)만 받고, 바이트는 `S3`로 직행한다.

이 결정으로 메인서버 메모리 사용량이 사용자 수에 따라 폭증하지 않는다. 동시 업로드 1,000건이 들어와도 메인서버 메모리는 거의 변하지 않는다.

11.5 아카이브 빌더 — 스트리밍 zip 생성

일괄 다운로드(7.5)에서 `zip`을 메모리에 다 올리면 사용자 한 명이 `1GB` 자료를 갖고 있을 때 `1GB` 메모리가 필요하다. 이를 피하려면 `zip`을 스트리밍으로 만든다.

```
fn build_archive(user_id, period):
    # 1. 임시 zip 파일을 디스크에 생성
    tmp_path = "/tmp/archive_{job_id}.zip"

    # 2. zip writer를 스트리밍 모드로 열기
    zw = open_zip_writer(tmp_path)

    # 3. 자료를 한 건씩 처리
    for asset in iter_user_assets(user_id, period):
        # S3에서 청크 단위로 읽기 (메모리 16MB 한계)
        with s3_streaming_read(asset.s3_key) as src:
            with zw.open_entry(asset.filename) as dst:
                copy_chunked(src, dst, chunk=4_MB)

    zw.close()

    # 4. 완성된 zip을 다시 S3로 멀티파트 업로드
    s3_multipart_upload(tmp_path, archive_s3_key)

    # 5. 임시 파일 삭제
    os.unlink(tmp_path)
```

이 방식의 메모리 사용은 사용자 자료 크기와 무관하게 고정 상수(체크 크기 × 동시성)로 유지된다.

11.6 ConnectionPool 재사용

Factory의 ConnectionPool 4개 커넥션 공유 패턴을 재사용한다. `mysql_ping` 실패 시 자동 복구 + `double-close` 방지도 그대로. 단 트래픽 규모가 다르므로 풀 크기는 8~16으로 시작하여 부하 테스트(12.9) 결과에 따라 조정한다.

11.7 메모리 워터마크 모니터링

프로세스 RSS를 1초 주기로 측정하여 워터마크를 두고, 임계치 초과 시 **HEALTH** 알림 발행 + 새 무거운 작업 거부.

- Soft Limit: 70% — 새 아카이브 작업 큐 적재 거부, 기존 작업은 진행
- Hard Limit: 85% — 모든 새 작업 거부
- Critical: 95% — graceful restart 트리거 (운영 정책)

12장. 트래픽 처리 + 캐시 다층 + 팬아웃

12.1 트래픽 발생 지점

이 시스템에서 트래픽이 폭증하는 지점은 다음과 같다.

- 피드 조회 — 사용자가 자기 피드를 새로고침할 때마다 발생. 가장 빈번.
- 팬아웃 — 인기 사용자가 글 발행 시 팔로워 N명에게 피드 갱신 필요.
- 미디어 다운로드 — 큰 파일 트래픽. CloudFront로 메인서버 우회.
- 시점 복원 — 무거운 연산. 캐시 필수.
- 의미 검색 — AI 허브 호출. 캐시 + 배치.

12.2 캐시 다층 구조

계층	구현	TTL	용도
L1 (인메모리)	C++ LRU 맵	30초	초고빈도 데이터: 사용자 세션, 본인 프로필
L2 (Redis)	Redis	5분	피드, 팔로우 관계, 임베딩 결과
L3 (DB)	MariaDB + 인덱스	영속	최종 진실
L4 (CDN)	CloudFront	1시간~	정적 미디어

12.3 캐시 무효화 전략

캐시는 "읽기는 캐시 우선, 쓰기는 즉시 무효화" 패턴(Cache-Aside + Write-Through 혼합)을 사용한다.

- posts INSERT/UPDATE/DELETE 시 → 작성자 본인 피드 캐시 무효화
- follows INSERT/DELETE 시 → 양쪽 사용자 피드 캐시 무효화

- 프로필 변경 시 → 본인 프로필 캐시 무효화 + 팔로워 피드의 작성자 정보 부분만 lazy 갱신

12.4 팬아웃 전략 — Push vs Pull

"인기 사용자가 글 발행 시 팔로워 N명에게 어떻게 피드를 갱신할 것인가"는 SNS 백엔드의 고전적 문제다. 두 전략의 트레이드오프:

전략	쓸 때	읽을 때	특징
Push (Fan-out on Write)	비용 큼 (N명에게 분배)	비용 작음 (자기 피드만 읽음)	팔로워 적은 사용자에게 유리
Pull (Fan-out on Read)	비용 작음 (자기 글만 저장)	비용 큼 (팔로우 N명 글 합치기)	팔로워 많은 사용자에게 유리
Hybrid	팔로워 수 임계치로 분기	임계치로 분기	실제 시스템(Twitter 등)이 채택

이 시스템은 Hybrid를 채택한다. 팔로워 1,000명 이하 사용자는 Push, 그 이상은 Pull.

12.5 팬아웃 워커

```

on POST_PERSISTED:
    publish FANOUT_REQUESTED { post_id, user_id, visibility }

worker FanoutWorker:
    on FANOUT_REQUESTED:
        if visibility == 'private':
            return # 팬아웃 안 함

        follower_count = follow_count(user_id)

        if follower_count <= 1000:
            # PUSH 전략
            for follower_id in followers(user_id):
                redis.lpush(f"feed:{follower_id}", post_id)
                redis.ltrim(f"feed:{follower_id}", 0, 999) # 최근 1000개만
        else:
            # PULL 전략 - 팔로워 피드 미리 갱신 안 함
            # 단, 작성자 자기 timeline에는 추가
            redis.lpush(f"timeline:{user_id}", post_id)
    
```

12.6 피드 조회

```

fn get_feed(viewer_id):
    # 1. PUSH로 받은 피드
    pushed = redis.lrange(f"feed:{viewer_id}", 0, 50)
    
```

```
# 2. PULL 대상(팔로워 1000명 초과 사용자)의 최신 글
pull_targets = follows_with_high_followers(viewer_id)
pulled = []
for target_id in pull_targets:
    pulled += redis.lrange(f"timeline:{target_id}", 0, 10)

# 3. 시간 정렬 + 권한 필터
merged = sort_by_time(pushed + pulled)
visible = filter_by_visibility(merged, viewer_id)

return visible[:50]
```

12.7 미디어 다운로드 트래픽

미디어 다운로드는 절대 메인서버를 거치지 않는다. CloudFront 서명 URL을 발급하면 클라이언트가 CloudFront에서 직접 받는다. 이 결정이 메인서버를 트래픽에서 보호하는 핵심이다.

12.8 Rate Limiting

대상	제한	근거
로그인 시도	IP당 분당 5회	brute force 방어
글 발행	사용자당 분당 10건	스팸 방어
미디어 업로드	사용자당 시간당 100건	스토리지 폭주 방어
AI 검색	사용자당 분당 30회	AI 허브 부하 통제
일괄 다운로드	사용자당 시간당 3건	워커 풀 보호

12.9 부하 테스트 계획

다음 시나리오로 k6 또는 wrk 부하 테스트를 수행할 예정이다. 시나리오 정의와 측정 후 결과는 15장에 기록한다.

- 시나리오 A: 동시 사용자 100~5000, 피드 조회 위주 (8:1:1 비율 — 조회:발행:검색)
- 시나리오 B: 인기 사용자(팔로워 10,000명)가 1초 1글 발행. 팬아웃 처리량 측정
- 시나리오 C: 동시 미디어 업로드 1,000건. 메인서버 메모리 비점유 검증

- 시나리오 D: 일괄 다운로드 동시 50건. Heavy Pool 큐잉·메모리 측정

각 시나리오에서 P50/P95/P99 응답 시간, 에러율, 메인서버 RSS, MariaDB CPU, Redis 메모리를 측정한다.

13장. API 명세

13.1 API 설계 원칙

- REST 기반, 자원 중심 URL
- 인증: JWT Access Token + Refresh Token
- 응답: 표준 JSON, 에러 시 problem+json
- 페이지네이션: cursor 기반 (created_at + id)
- idempotency: 쓰기 요청에 Idempotency-Key 헤더 옵션 지원

13.2 인증

메서드	경로	설명
POST	/auth/signup	회원가입
POST	/auth/login	로그인 → access + refresh 발급
POST	/auth/refresh	refresh로 access 재발급
POST	/auth/logout	refresh 무효화

13.3 사용자 / 팔로우

메서드	경로	설명
GET	/me	본인 프로필
PATCH	/me	본인 프로필 수정
GET	/users/{id}	타인 프로필 (공개 정보만)
POST	/users/{id}/follow	팔로우
DELETE	/users/{id}/follow	언팔로우
GET	/me/followers	본인 팔로워 목록

GET	/me/following	본인 팔로잉 목록
-----	---------------	-----------

13.4 글 / 미디어

메서드	경로	설명
POST	/posts	글 발행 (visibility, download_policy 포함)
GET	/posts/{id}	글 조회 (권한 체크)
PATCH	/posts/{id}	글 수정 (이벤트 추가)
DELETE	/posts/{id}	글 삭제 (이벤트 추가, 실제 행은 soft delete)
POST	/uploads	S3 멀티파트 서명 URL 발급
POST	/uploads/{id}/complete	업로드 완료 통지
GET	/media/{id}/view	조회용 서명 URL 발급
GET	/media/{id}/download	다운로드용 서명 URL 발급 (권한 체크)

13.5 피드 / 검색 / 시점

메서드	경로	설명
GET	/me/feed	본인 피드 (Push+Pull 혼합)
GET	/me/timeline	본인이 쓴 글 타임라인
GET	/users/{id}/timeline	타인 타임라인 (권한 필터)
GET	/me/search?q=...	의미 기반 검색
GET	/me/snapshot?at=...	임의 시점 상태 복원
POST	/me/compare	두 시점 비교 (LLM 자연어 설명)

13.6 본인 자료 아카이브

메서드	경로	설명
POST	/me/archives	일괄 다운로드 작업 생성

GET	/me/archives/{id}	작업 상태 조회
GET	/me/archives/{id}/download	완성 시 서명 URL 발급

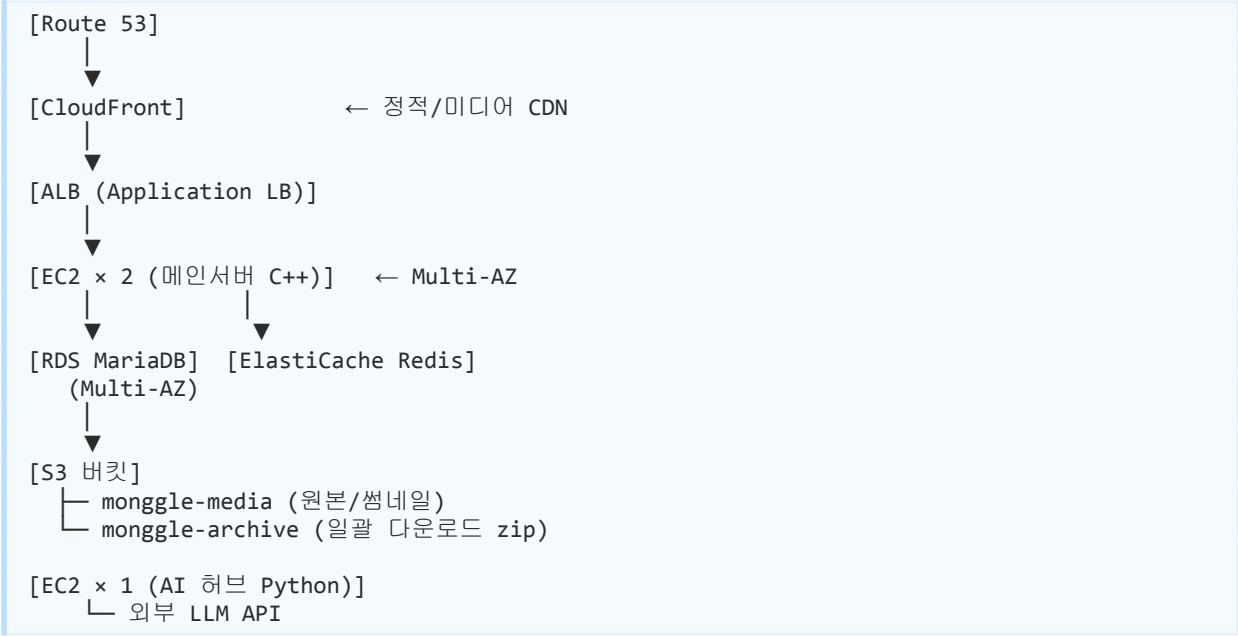
13.7 에러 응답 형식

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json

{
  "type": "https://monggle.local/errors/forbidden",
  "title": "Forbidden",
  "status": 403,
  "detail": "이 글의 다운로드 권한이 없습니다.",
  "instance": "/media/12345/download",
  "trace_id": "abcd-1234"
}
```

14장. AWS 인프라 운영 계획

14.1 인프라 구성도



14.2 EC2 인스턴스

역할	타입	OS	근거
메인서버	c6i.xlarge × 2	Ubuntu 24 LTS	CPU 우선 워크로드, Factory와 동일 OS
AI 허브	c6i.large × 1	Ubuntu 24 LTS	BGE-m3 CPU 추론

14.3 RDS

- 엔진: MariaDB
- 인스턴스: db.r6g.large (메모리 우선)
- Multi-AZ: 활성화
- 자동 백업: 7일
- 스냅샷: 일별, 30일 보관

14.4 S3 정책

버킷	퍼블릭	암호화	Lifecycle
monggle-media	비공개 (CloudFront만)	AES-256	30일 후 IA, 90일 후 Glacier (선택)
monggle-archive	비공개 (서명 URL만)	AES-256	7일 후 자동 삭제
monggle-uploads-pending	비공개	AES-256	1일 후 미완료 업로드 정리

14.5 CloudFront

- Origin: monggle-media S3 버킷
- 서명 URL/쿠키 활성화 (퍼블릭 캐싱 안 함)
- 캐시 키: 파일 키 + 서명 만료 시각
- OAI(Origin Access Identity)로 S3 버킷 직접 접근 차단

14.6 보안 — Defense in Depth

Factory의 Defense in Depth 원칙을 그대로 적용한다.

- Prepared Statement (모든 DAO)
- 입력 크기 제한 (12.2)
- Rate Limiting (13.8)
- bcrypt + Salt 비밀번호
- JWT 서명 검증 (HS256 또는 RS256)
- Path Traversal 차단 (S3 키 생성 시 사용자 입력 직접 사용 X, 서버 측 UUID 사용)
- JSON Injection 방지 — 모든 사용자 입력 escape

- CORS 화이트리스트 (프론트 도메인만)

14.7 모니터링

항목	도구	알람 조건
메인서버 RSS	CloudWatch Custom	Hard Limit 85% 이상
EventBus 큐 깊이	CloudWatch Custom	HW 10,000 도달
RDS CPU	CloudWatch	80% 5분 지속
RDS 커넥션	CloudWatch	풀 한계의 90%
Redis 메모리	CloudWatch	75% 도달
AI 허브 응답 지연	CloudWatch Custom	P95 > 2초
LLM 외부 API 실패율	CloudWatch Custom	5분간 10% 초과

14.8 로그

Factory의 "한글 행동 중심 로그 + 자정 자동 로테이션"을 동일하게 적용한다. 추가로 CloudWatch Logs로 중앙 수집, `trace_id`로 요청 단위 추적.

14.9 배포

- 메인서버: Blue/Green (ALB Target Group 전환)
- AI 허브: Rolling (요청량 적음)
- DB 마이그레이션: 무중단 우선 (ALTER 분리, online schema change)
- 이벤트 스키마 변경: **backward compatible** 강제, 한 버전 유지 후 제거

15장. 부하 테스트 계획

15.1 부하 테스트 시나리오 (Week 4 진행 예정)

Week 4에 다음 시나리오로 k6 또는 wrk 부하 테스트를 진행할 예정이다. 측정 후 본 표를 실제 수치로 채운다. 시스템 어필 측이 시점 복원·자원 설계·트래픽 처리이므로, 시나리오도 이 세 축을 검증할 수 있게 구성했다.

시나리오	동시 사용자	RPS	P50	P95	P99	에러율	병목
A1 — 피드 100명	100	예정	예정	예정	예정	예정	예정
A2 — 피드 1,000명	1,000	예정	예정	예정	예정	예정	예정
A3 — 피드 5,000명	5,000	예정	예정	예정	예정	예정	예정
B — 인기 팬아웃	—	예정	예정	예정	예정	예정	예정
C — 동시 업로드	1,000	예정	예정	예정	예정	예정	예정
D — 일괄 다운로드	50	예정	예정	예정	예정	예정	예정

15.2 측정 항목

- 응답 시간: P50 / P95 / P99 (ms)
- 처리량: RPS (requests per second)
- 에러율: 4xx, 5xx, timeout 비율
- 자원 사용: 메인서버 RSS·CPU, RDS CPU·커넥션, Redis 메모리, AI 허브 응답 시간
- EventBus 큐 깊이 추이

15.3 예상 병목과 대응

예상 병목	징후	대응
RDS CPU	P95 > 500ms, 쿼리 대기	쿼리 최적화, 인덱스 추가, Read Replica
메인서버 메모리	RSS 워터마크 도달	스트리밍 zip 검증, 풀 크기 조정
EventBus 큐	HW 도달, 외부 큐 우회 발생	워커 수 증가, 핫 스팟 분리
AI 허브	임베딩 응답 지연	캐시 적중률 점검, 배치 크기 조정
LLM 외부 API	rate limit 도달	회로 차단, 캐시 TTL 연장, fallback 메시지